🎨 Свои текстуры для кнопок меню

Тут описано:

- как подменить скучные кнопки из `uiskin.json` на свою PNG-текстуру;
- как работает текущая раскладка кнопок «в столбик с паддингами»;
- как сдвинуть кнопки вправо;
- как разместить кнопки в **точных координатах** на экране;
- как положить на кнопку текст;
- как в целом загружать любые картинки/спрайты в игру (не только кнопки).

Всё по шагам, с готовым кодом.

📚 Содержание

1. Как сейчас устроены кнопки
2. Шаг 1 — кладём PNG в правильную папку
3. Шаг 2 — загружаем текстуру и собираем стиль
4. Шаг 3 — учим GameButton принимать наш стиль
5. Шаг 4 — применяем стиль к кнопкам меню
6. Как текст оказывается поверх картинки
7. Как сделать кнопку больше/меньше
8. Как двигать кнопки вправо
9. Как разместить кнопки по точным координатам
10. Бонус: не только кнопки — работа с любыми спрайтами
11. Типичные ошибки и чек-лист

1. Как сейчас устроены кнопки

Открой `MainMenuScreen.kt` и найди блок в `show()`:

```
root.add(btnPlay).pad(12f).row()
root.add(btnRules).pad(12f).row()
root.add(btnSettings).pad(12f).row()
root.add(btnExit).pad(12f).row()
```

Что здесь происходит — по шагам, слева направо:

Кусок	Что делает

Кусок	Что делает
`root.add(btnPlay)`	кладёт кнопку в **ячейку** таблицы `root`. Пока это просто «слот», размер которого зависит от кнопки и паддинга
`.pad(12f)`	отступ **вокруг** ячейки в **виртуальных координатах** (у нас мир 900×450). 12 — небольшая щёлочка между кнопками
`.row()`	«перенос строки». Следующий `add(...)` окажется **под** этой кнопкой, а не справа

Дальше таблица `root`:

- `root.setFillParent(true)` → занимает **весь экран**;
- по умолчанию ячейки **центрируются** — поэтому столбик кнопок стоит по центру;
- одна кнопка = одна строка таблицы, сверху вниз.

Порядок = строки. Если переставить `root.add(...)` местами — кнопки поменяются местами. Убрать `.row()` — следующая кнопка встанет справа от предыдущей.

> 💡 **Аналогия:** `Table` — это как блокнот в клеточку. `add()` — кладёшь предмет в клетку, `.row()` — переходишь на следующую строку. Всё, что попало в таблицу, автоматически центрируется/выравнивается по её правилам.

2. Шаг 1 — кладём PNG в правильную папку

Все ассеты живут в **`android/assets/`** (это «корень» для `Gdx.files.internal`). Создай там папку `buttons/` и положи файл:

```
android/
  assets/
    buttons/
      menubutton.png   ← твоя текстура
    main-menu1.png
    ui/
      uiskin.json
      ...
```

В коде обращаешься по **относительному пути от `assets/`**, без `android/assets/`:

```
Gdx.files.internal("buttons/menubutton.png")
```

> ⚠️ **Требования к картинке:** PNG, желательно одинаковой ширины/высоты у всех кнопок (у нас одна текстура — тем проще). Обычный размер под наш мир 900×450 — примерно **240×70** пикселей. Слишком маленькую картинку придётся растягивать — будет мыло.

3. Шаг 2 — загружаем текстуру и собираем стиль

libGDX не рисует кнопку «просто из текстуры» — ему нужен **стиль** (`TextButtonStyle`). Стиль — это набор *«как выглядит кнопка в разных состояниях + каким шрифтом писать текст»*.

Состояния кнопки:

Поле стиля	Когда используется
`up`	кнопка в покое
`down`	кнопку нажали
`over`	курсор наведён (на десктопе)
`checked`	для «переключателей» (нам пока не надо)
`font`	шрифт текста
`fontColor`	цвет текста в покое
`downFontColor`	цвет текста при нажатии

Открой `Main.kt` и перепиши `create()` + `dispose()` :

```
package ru.triplethall.mushroomhunt

import com.badlogic.gdx.Game
import com.badlogic.gdx.Gdx
import com.badlogic.gdx.graphics.Color
import com.badlogic.gdx.graphics.GL20
import com.badlogic.gdx.graphics.Texture
import com.badlogic.gdx.scenes.scene2d.ui.Skin
import com.badlogic.gdx.scenes.scene2d.ui.TextButton
import com.badlogic.gdx.scenes.scene2d.utils.TextureRegionDrawable

class Main : Game() {

    lateinit var skin: Skin

    // Держим текстуру кнопки отдельным полем — чтобы потом её dispose()
    lateinit var buttonTexture: Texture

    override fun create() {
        skin = Skin(Gdx.files.internal("ui/uiskin.json"))

        // 1. Загружаем картинку кнопки
        buttonTexture = Texture(Gdx.files.internal("buttons/menubutton.png"))

        // 2. Собираем стиль
        val menuStyle = TextButton.TextButtonStyle()

        // up = обычный вид
        menuStyle.up = TextureRegionDrawable(buttonTexture)
```

```
        // down = нажатый вид (слегка затемнённая копия)
        menuStyle.down = TextureRegionDrawable(buttonTexture)
            .tint(0.75f, 0.75f, 0.75f, 1f)

        // over = при наведении мыши (опционально, но приятно)
        menuStyle.over = TextureRegionDrawable(buttonTexture)
            .tint(1.1f, 1.1f, 1.1f, 1f)

        // 3. Шрифт берём из уже загруженного скина
        menuStyle.font = skin.getFont("default-font")

        // 4. Цвет текста. Если у тебя СВЕТЛАЯ текстура — поставь тёмный цвет,
        //    например Color(0.1f, 0.1f, 0.1f, 1f)
        menuStyle.fontColor = Color.WHITE

        // 5. Регистрируем стиль в скине под именем "menu"
        skin.add("menu", menuStyle)

        setScreen(MainMenuScreen(this))
    }

    override fun render() {
        Gdx.gl.glClearColor(0.06f, 0.07f, 0.08f, 1f)
        Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT)
        super.render()
    }

    override fun dispose() {
        skin.dispose()
        buttonTexture.dispose() // не забываем прибраться!
    }
}
```

Что важно понять:

- **`TextureRegionDrawable`** — «обёртка», которая говорит libGDX: *«вот тут Texture, рисуй его»*. `up` / `down` / `over` — именно `Drawable`, а не `Texture`.
- **`tint(r, g, b, a)`** — умножает цвет пикселей. Все каналы `0.75` → картинка темнее в 0.75 раз. Так из одной текстуры получаем «нажатую» и «наведённую» версии.
- **`skin.add("menu", menuStyle)`** — кладём стиль в скин. Теперь по имени `"menu"` можно его достать: `skin.get("menu", TextButton.TextButtonStyle::class.java)`.

4. Шаг 3 — учим GameButton принимать наш стиль

У `TextButton` (родителя нашей `GameButton`) есть **три** полезных конструктора:

1. `TextButton(text, skin)` — берёт стиль по умолчанию из скина;
2. `TextButton(text, skin, styleName)` — берёт стиль по имени (например, `"menu"`);

3. `TextButton(text, style)` — берёт стиль напрямую.

Нам нужен вариант **2**, но аккуратно: старый код `GameButton("PLAY", game.skin)` должен продолжать работать (попап же использует его!). Поэтому добавим параметр `styleName` со значением по умолчанию `"default"`:

```
// GameButton.kt
package ru.triplethall.mushroomhunt

import com.badlogic.gdx.scenes.scene2d.InputEvent
import com.badlogic.gdx.scenes.scene2d.ui.Skin
import com.badlogic.gdx.scenes.scene2d.ui.TextButton
import com.badlogic.gdx.scenes.scene2d.utils.ClickListener

class GameButton(
    text: String,
    skin: Skin,
    styleName: String = "default"   // ← новое
) : TextButton(text, skin, styleName) {

    var onClick: () -> Unit = {}

    init {
        addListener(object : ClickListener() {
            override fun clicked(event: InputEvent?, x: Float, y: Float) {
                onClick()
            }
        })
    }
}
```

Теперь:

- `GameButton("CLOSE", skin)` — как раньше, стиль `"default"` (тот, что в `uiskin.json`);
- `GameButton("PLAY", skin, "menu")` — наша новая текстура.

> ⚠️ **Если стиля `"default"` в твоём `uiskin.json` нет**, конструктор упадёт. Тогда вместо дефолта подставь имя реально существующего стиля, например `"default"` → `"text-button"`. Проверить можно, открыв `uiskin.json` и поискав секцию `com.badlogic.gdx.scenes.scene2d.ui.TextButton$TextButtonStyle`.

5. Шаг 4 — применяем стиль к кнопкам меню

В `MainMenuScreen.kt`:

```
val btnPlay     = GameButton("PLAY",     game.skin, "menu")
val btnRules    = GameButton("RULES",    game.skin, "menu")
val btnSettings = GameButton("SETTINGS", game.skin, "menu")
val btnExit     = GameButton("EXIT",     game.skin, "menu")
```

Всё остальное (слушатели, `.pad(12f).row()`) — **не трогаем**. Кнопки в меню теперь с твоей картинкой. 🎉

Попап не трогаем: там `GameButton("CLOSE", skin)` — останется на старом стиле. Если хочешь и там новую — передай третьим аргументом `"menu"`.

6. Как текст оказывается поверх картинки

Сам по себе — **магически**. `TextButton` устроен так:

1. Рисует **фон** из `up` / `down` / `over` (наш PNG);
2. Поверх — **текст**, переданный в конструктор, по центру.

Ничего дополнительно делать не надо. Управляется через стиль:

```
menuStyle.font = skin.getFont("default-font")  // какой шрифт
menuStyle.fontColor = Color.WHITE              // цвет в покое
menuStyle.downFontColor = Color.LIGHT_GRAY     // цвет при нажатии
```

Если текст «сливается» с фоном:

- **тёмная текстура** → `fontColor = Color.WHITE`;
- **светлая текстура** → `fontColor = Color(0.1f, 0.1f, 0.1f, 1f)`;
- хочешь обводку/тень — это уже через `BitmapFont` и `LabelStyle`, отдельная история.

Размер шрифта берётся из `.fnt`-файла, на который ссылается скин. Если текст мелкий — либо возьми в скине шрифт покрупнее (например, `"big-font"`, если он есть), либо сгенерируй свой `.fnt` через **Hiero** (утилита из libGDX).

> 💡 Хочешь «попадание» текста идеально по центру — на самой текстуре **не рисуй** никаких надписей/иконок, оставь середину пустой. `TextButton` центрирует текст по bounds кнопки, а не по «глазу».

7. Как сделать кнопку больше/меньше

По умолчанию размер кнопки = **`max(минимальный размер картинки, минимальный размер текста)`**. То есть если картинка 240×70 — кнопка будет не меньше 240×70.

Управлять размером можно **в таблице** (через ячейку):

```
root.add(btnPlay).width(300f).height(90f).pad(12f).row()
```

или сразу **на актёре** (если добавляешь не в таблицу, а на сцену — см. раздел 9):

```
btnPlay.setSize(300f, 90f)
```

> ⚠️ Если картинка маленькая (например, 100×30), а ты тянешь кнопку до 300×90 — текстура **растянется** и поплывёт. Для красивых «скруглённых» кнопок с произвольным размером используют `NinePatch` — это текстура, у которой края не растягиваются. Пока не заморачивайся, но знай, что такая штука есть.

8. Как двигать кнопки вправо

Есть **два** принципиально разных подхода. Выбирай по задаче.

🅰️ Подход через `Table` (кнопки остаются «в столбик», но столбик сдвинут вправо)

Способ 1 — выровнять всё содержимое `root` вправо:

```
root.right()             // прижать содержимое к правому краю
root.padRight(80f)       // плюс отступ 80 от правого края
```

`root.right()` = `root.align(Align.right)` — все ячейки теперь жмутся вправо.

Способ 2 — обернуть кнопки в отдельную таблицу-«колонку» и выровнять её:

```
val column = Table()
column.add(btnPlay).pad(12f).row()
column.add(btnRules).pad(12f).row()
column.add(btnSettings).pad(12f).row()
column.add(btnExit).pad(12f).row()

// Кладём колонку в root. expand() = занять всё место,
// right() = прижать колонку вправо, padRight = отступ от края
root.add(column).expand().right().padRight(80f)
```

Что тут важно:

- `root.add(column)` — создаёт **одну** ячейку на весь экран;
- `.expand()` — ячейка растягивается на всё доступное место (без него колонка стояла бы по центру);

- `.right()` — **внутри ячейки** содержимое прижимается вправо;
- `.padRight(80f)` — 80 виртуальных единиц от правого края.

Способ 2 — самый гибкий: можно оставить одну часть контента слева, другую справа, добавить заголовок сверху и т.д.

🅱️ Подход через координаты (см. следующий раздел)

Если кнопки нужны не «столбиком справа», а «в произвольных местах» — таблица не нужна.

> 🧠 **Проверь себя:** в чём разница `.right()` и `.padRight(...)`?
>
> - `.right()` — выравнивание **содержимого внутри ячейки**;
> - `.padRight(n)` — **отступ** от края ячейки/таблицы.
>
> Можно комбинировать: `.right().padRight(80f)` — «прижать вправо и отодвинуть на 80».

9. Как разместить кнопки по точным координатам

Таблицу `root` тут **не используем** (или оставляем пустой). Каждую кнопку кладём **прямо на сцену**:

```
override fun show() {
    Gdx.input.inputProcessor = stage

    val btnPlay     = GameButton("PLAY",     game.skin, "menu")
    val btnRules    = GameButton("RULES",    game.skin, "menu")
    val btnSettings = GameButton("SETTINGS", game.skin, "menu")
    val btnExit     = GameButton("EXIT",     game.skin, "menu")

    // Навешиваем действия — как раньше
    btnPlay.onClick     = { PlayAction(game, stage).run() }
    btnRules.onClick    = { RulesAction(game, stage).run() }
    btnSettings.onClick = { SettingsAction(game, stage).run() }
    btnExit.onClick     = { ExitAction(game, stage).run() }

    // Размер и позиция каждой кнопки
    // setSize(ширина, высота)
    // setPosition(x, y) — это ЛЕВЫЙ НИЖНИЙ угол актёра в координатах мира
    place(btnPlay,     590f, 300f)
    place(btnRules,    590f, 220f)
    place(btnSettings, 590f, 140f)
    place(btnExit,     590f,  60f)

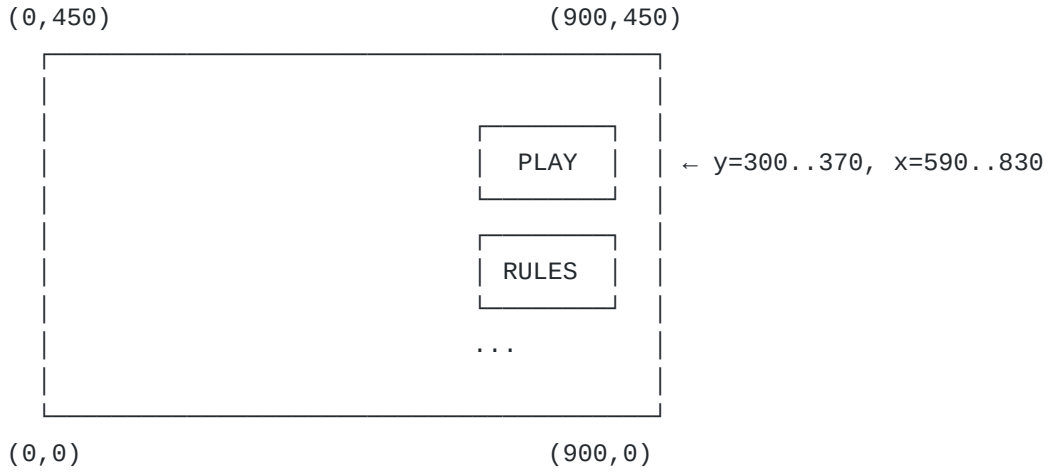
    stage.addActor(btnPlay)
    stage.addActor(btnRules)
    stage.addActor(btnSettings)
    stage.addActor(btnExit)
}

private fun place(button: GameButton, x: Float, y: Float) {
```

```
    button.setSize(240f, 70f)
    button.setPosition(x, y)
}
```

🧭 Координаты — как читать

- Мир у нас **900×450** (`FitViewport(900f, 450f, camera)`).
- Начало координат `(0, 0)` — **нижний левый угол** (не верхний, как в веб-вёрстке!).
- `x` растёт **вправо**, `y` — **вверх**.
- Правый верхний угол = `(900, 450)` .
- Кнопка размером `240×70` в позиции `(590, 300)` займёт прямоугольник `x=590..830, y=300..370` .



🎯 Полезные приёмы

Центрировать кнопку по горизонтали:

```
val cx = (900f - 240f) / 2f   // = 330
btnPlay.setPosition(cx, 300f)
```

Прижать к правому краю с отступом 40:

```
btnPlay.setPosition(900f - 240f - 40f, 300f)   // x = 620
```

Адаптивность при смене размера окна: координаты выше **жёсткие** — если игрок развернёт окно, viewport пересчитает масштаб, но относительное расположение кнопок в мире не поменяется (что обычно и хорошо — они «прибиты» к правому краю мира).

> ⚠️ **Порядок `stage.addActor` = порядок рисования.** Кто добавлен позже, тот **сверху**. Поэтому попап (`InfoPopup`), добавленный при клике, окажется над кнопками — это то, что нам надо.

> ⚠️ **Кнопки прямо на сцене не получают автоматический размер.** Обязательно

> `setSize(...)` (или `button.pack()` — тогда размер возьмётся из текста+картинки по минимуму). Без этого кнопка будет нулевой и кликам станет некуда попадать.

10. Бонус: не только кнопки — работа с любыми спрайтами

Всё, что ты сейчас узнал про кнопки, — это **частный случай** общей схемы работы с ассетами в libGDX. Запомни универсальный конвейер:

```
PNG-файл  →  Texture  →  Drawable (или TextureRegion / Sprite)  →  Actor  →  Stage
```

10.1. Загрузка и владение

```
// Владелец = тот, кто создал. Он же и dispose().
val texture = Texture(Gdx.files.internal("sprites/mushroom.png"))
```

Правило: **кто `new Texture` — тот и `dispose()`**. Не dispose'ишь — утечка видеопамяти. Dispose'ишь дважды — крэш.

Жизненный цикл для ресурсов экрана — в `dispose()` этого экрана:

```
override fun dispose() {
    texture.dispose()
    batch.dispose()
    // ...
}
```

10.2. Показать картинку как актёра

`Image` — самый простой актёр-картинка:

```
val mushroom = Image(Texture(Gdx.files.internal("sprites/mushroom.png")))
mushroom.setSize(64f, 64f)
mushroom.setPosition(100f, 200f)
stage.addActor(mushroom)
```

`Image` принимает `Texture`, `TextureRegion`, `Drawable` или `NinePatch`. Кладётся и в `Table`, и на `Stage`.

В таблице:

```
root.add(Image(texture)).size(64f, 64f).pad(8f)
```

10.3. TextureRegion — кусок текстуры

Если у тебя **спрайт-лист** (много картинок в одном PNG), берёшь «окошки»:

```
val sheet = Texture(Gdx.files.internal("sprites/mushrooms.png"))
val redMushroom = TextureRegion(sheet, 0, 0, 64, 64)        // x, y, w, h в пикселях
val blueMushroom = TextureRegion(sheet, 64, 0, 64, 64)
val yellowMushroom = TextureRegion(sheet, 128, 0, 64, 64)

stage.addActor(Image(redMushroom))
```

Координаты `x, y` региона — **в пикселях текстуры**, от **левого верхнего** угла (у `TextureRegion` другая система — не путай с актёрами).

10.4. Своя анимация и спрайт

Для анимаций нужен `Animation<TextureRegion>`:

```
val frames = arrayOf(
    TextureRegion(sheet, 0, 0, 64, 64),
    TextureRegion(sheet, 64, 0, 64, 64),
    TextureRegion(sheet, 128, 0, 64, 64),
)
val walk = Animation(0.1f, frames)  // 0.1с на кадр
```

`Animation` рисуют вручную в `render()` через `batch` — но это уже отдельная тема, не для меню.

10.5. Sprite и SpriteBatch

`SpriteBatch` — «художник» для быстрой пакетной отрисовки. Уже используется у нас для фона:

```
batch.begin()
batch.draw(texture, x, y, width, height)
batch.end()
```

`Sprite` — это `TextureRegion` + координаты + цвет + поворот. Удобно, когда спрайтов много и они двигаются:

```
val sprite = Sprite(texture)
sprite.setPosition(200f, 150f)
sprite.setSize(64f, 64f)

// В render():
batch.begin()
sprite.draw(batch)
batch.end()
```

Для UI-меню тебе хватит `Image`, а `Sprite` / `SpriteBatch` пригодятся в игровом экране.

10.6. Хранение ассетов

Пока ассетов мало — держим их полями в `Main`/экране. Как вырастет проект — используют **`AssetManager`** (ленивая загрузка + прогресс-бар). Это на будущее:

```
val manager = AssetManager()
manager.load("buttons/menubutton.png", Texture::class.java)
manager.finishLoading()
val tex = manager.get("buttons/menubutton.png", Texture::class.java)
manager.dispose()   // снесёт всё разом
```

10.7. Стили в скине — не только кнопки

`skin` — это **база именованных стилей**. В `uiskin.json` лежат стили не только для `TextButton`, но и для `Label`, `TextField`, `ImageButton` … Принцип тот же, что мы применили:

```
val labelStyle = Label.LabelStyle()
labelStyle.font = skin.getFont("default-font")
labelStyle.fontColor = Color.GOLD
skin.add("gold-label", labelStyle)

val label = Label("Score: 42", skin, "gold-label")   // ← стиль по имени
```

Мораль: **любой визуал в libGDX идёт через Style**, собранный из `Drawable` + шрифтов + цветов. Один раз понял — работает везде.

11. Типичные ошибки и чек-лист

🐞 «Кнопка не отображается / нулевого размера»

При добавлении **напрямую на сцену** забыл `setSize(...)`. Добавь:

```
btn.setSize(240f, 70f)
```

При добавлении в `Table` размер обычно берётся из текстуры/текста, но проверить не помешает: `.width(w).height(h)`.

🐞 «Кнопка есть, но клики не работают»

- В `show()` не выставил `Gdx.input.inputProcessor = stage`. У нас выставлен — но если переписывал, не забудь.
- Кнопку закрыл другой прозрачный актёр сверху (например, `Table` с `setFillParent(true)` без `Touchable.childrenOnly`). Проверь `touchable` у «пустых» таблиц.
- Кнопка **под** попапом — тогда сначала закрой попап.

🐞 «Текстуры нет / FileNotFound»

- Лежит **не в `android/assets/`** . Положи туда.
- В коде указан **абсолютный** путь или с `android/assets/...` в начале. Нужно от `assets/` : `"buttons/menubutton.png"` .
- Опечатка в имени (регистр важен: `MenuButton.PNG ≠ menubutton.png`).

🐞 «Текст не видно на кнопке»

Цвет текста совпал с цветом текстуры. Поставь контрастный `fontColor` .

🐞 «После выхода из игры крэш / артефакты»

Забыл `dispose()` на `Texture` / `Skin` / `SpriteBatch` . Проверь `Main.dispose()` и `MainMenuScreen.dispose()` .

🐞 «Стиль `"menu"` не найден»

`skin.add("menu", menuStyle)` не выполнился **до** создания кнопок. Регистрация — в `Main.create()` **до** `setScreen(...)` .

🐞 «Кнопка не двигается, как хочу»

Убедись, что кнопка **не в `Table`** — таблица перезапишет `setPosition` своим лэйаутом. Если хочешь точные координаты — только `stage.addActor(btn)` .

✅ Чек-лист «я всё сделал правильно»

- [ ] PNG лежит в `android/assets/buttons/menubutton.png`
- [ ] В `Main.create()` загружен `Texture` , собран `TextButtonStyle` , добавлен в `skin` под именем `"menu"`
- [ ] `GameButton` принимает третий аргумент `styleName`
- [ ] В `MainMenuScreen` кнопки создаются с `"menu"`
- [ ] В `Main.dispose()` есть `buttonTexture.dispose()`
- [ ] Если двигал вправо — понимаю разницу `.right()` / `.padRight()`
- [ ] Если по координатам — у каждой кнопки есть `setSize(...)` и `setPosition(...)` , они добавлены прямо на `stage`
- [ ] Текст читается на фоне (контрастный `fontColor`)

🏁 Что попробовать самому

1. **Поменяй цвет текста** на кнопке: сделай его золотым (`Color.GOLD`). Что изменится?
2. **Сдвинь столбик кнопок вправо** через `root.right().padRight(60f)` — добейся, чтобы столбик стоял «у правого края».
3. **Сделай кнопки разного размера**: `PLAY` — 260×90, остальные — 220×60. Через `.width(w).height(h)` в таблице.
4. **Размести кнопки по координатам**: PLAY в правом верхнем углу, EXIT — в левом нижнем. Подсказка: `x = 900f - w - отступ , y = 0 + отступ .`
5. **Сделай «нажатую» кнопку заметнее**: в стиле `down` сделай `tint(0.5f, 0.5f, 0.5f)` — сильно темнее.
6. **Добавь картинку-логотип над кнопками**: `Image(Texture("ui/logo.png"))` через `stage.addActor` с координатами.

Удачи!